CSC 648 / CSC 848

Section 03

Milestone 3 — Version 2 (M3v2)

Project: **Home4U**
AI-Assisted Interior Design Recommendation Platform

Team Number: **4**
Team Alias: **Vibecoding for Internship**

Team Members and Roles

Caleb Ponce — Team Lead, Backend/Auth, Deployment
Mason Lee — Frontend/UI
Christopher Quach — Backend/API & Database Alignment
Tyler Morris — QA / Smoke Tests / Deployment Verification
Dias Almat — Data (Styles/Tags/Search) & Recommendations

Team Lead Email

cponce8@sfsu.edu

Date

April 14, 2026

Revision History

Version	Date	Author(s)	Description
M3v2	April 21, 2026	Team	Redid diagrams to match frontend designs. Reworked wireframes.
M3v1	April 14th, 2026	Dias, Team	Added diagrams, design patterns
M2v2	Mar 5th, 2026	Caleb Ponce	Revised Milestone 2 submission, edited the team contributions/scores, made sure it met demands, and submitted a new one.
M2v1	Mar 4, 2026	Team	Final Milestone 2 submission
M2v0.9	Mar 3, 2026	Team	Completed architecture diagrams, APIs, and deployment documentation
M2v0.7	Mar 2, 2026	Team	Drafted data definitions and functional requirements
M2v0.5	Mar 1, 2026	Team	Created Milestone 2 document structure
M1v2	Feb 19, 2026	Team	Revised Milestone 1 based on instructor feedback
M1v1	Feb 8, 2026	Team	Initial Milestone 1 submission
M1v0.1	Feb 1, 2026	Team	Initial project draft

Table of Contents

1. Data Definitions

User

Represents a registered user of the platform.

Attributes

user_id
email
password_hash
created_at

Relationship:

User → RoomProject

RoomProject

Represents a user-created room design project.

Attributes

project_id
user_id (FK → User)
room_type
budget
photo_url
created_at

Relationship:

RoomProject → RoomTag
RoomProject → Room Dimension
RoomProject → ResemblanceScore
RoomProject → Recommendation
RoomProject → RoomFurniture
RoomProject → RoomObject

Style

Represents an interior design style available for recommendation.

Attributes

style_id
name
description

Relationship:
Style → StyleTag
Style → ProductItem
Style → ResemblanceScore

Tag

Represents descriptive keywords used to categorize styles.

Attributes

tag_id
name

Relationship:
Tag → RoomTag
Tag → StyleTag

StyleTag

Associates tags with styles.

Attributes

styletag_id
style_id
tag_id
weight

RoomTag

Associates tags with a room project.

Attributes

roomtag_id
room_project_id
tag_id
is_confirmed

ResemblanceScore

Stores similarity scores between room projects and styles.

Attributes

resemblancescore_id
room_project_id
style_id
score_value
created_at

Recommendation

Stores design recommendations generated for a project.

Attributes

recommendation_id
room_project_id
description
priority_score
estimated_cost
is_completed
created_at

Relationship:
Recommendation → RoomProject

ProductItem

Represents product suggestions associated with a style.

Attributes

product_id
style_id
vendor_id
name
category
estimated_cost
product_url
Image_url

Relationship:
ProductItem → Style
ProductItem → Vendor
ProductItem → BudgetAllocation

BudgetPlan

Represents the budget associated with the project

Attributes

budget_id
project_id
total_budget
currency
plan_name
created_at

Relationship:
BudgetPlan → RoomProject
BudgetPlan → BudgetAllocation

BudgetAllocation

Represents how the budget is being spent

Attributes

allocation_id

budget_id
product_id
category
allocated_amount
quantity
priority_rank
created_at

Relationship:
BudgetAllocation → ProductItem
BudgetAllocation → BudgetPlan

---

2. Prioritized Functional Requirements

Priority 1 — Critical

FR1 — User Signup
Users shall be able to register using an email and password.

FR2 — User Login
Users shall authenticate and receive a JWT token.

FR3 — Create Room Project
Authenticated users shall create room design projects.

FR4 — List Room Projects
Users shall retrieve projects associated with their account.

FR5 — Retrieve Styles
Users shall retrieve available interior design styles.

FR6 — Retrieve Style Tags
Users shall retrieve tags associated with styles.

FR7 — Search Styles
Users shall search styles using text queries.

FR8 — Generate Recommendations
Users shall receive recommendations associated with a project.

FR9 — Enforce Project Ownership
The system shall ensure that users can access only the room projects associated with their own account.

FR10 — Validate User Credentials
The system shall validate user credentials before granting access to protected resources.

FR11 — Persist Room Project Data
The system shall store room project information in the database for later retrieval.

FR12 — Associate Recommendations with Styles
The system shall associate each generated recommendation with a style.

Priority 2 — Important

FR13 — Update Room Project
Users shall modify project information.

FR14 — Delete Room Project
Users shall delete projects.

FR15 — Upload Room Photo
Users shall upload a room image which is stored locally.

FR16 — Assign Tags to Room Project
Users shall associate tags with projects.

FR17 — Compute Resemblance Scores
The system shall calculate similarity scores between room tags and style tags.

FR18 — Mark Recommendation Complete
Users shall mark recommendations as completed.

FR19 — Update Recommendation Status
Users shall update the status of a recommendation.

FR20 — Retrieve Tags
Users shall retrieve the set of available tags.

FR21 — Search Tags
Users shall search tags using text queries.

FR22 — Associate Style Tags with Styles
The system shall associate tags with styles.

FR23 — Support Multiple Room Photos per Project
The system shall allow a room project to have multiple uploaded room photos.

Priority 3 — Nice to Have

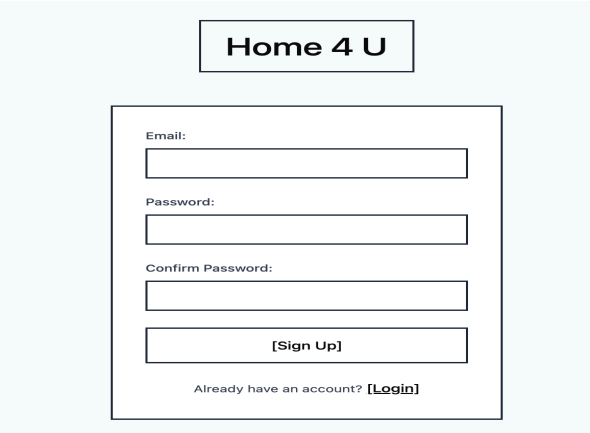
FR24 — Product Suggestions
The system may recommend purchasable furniture products.

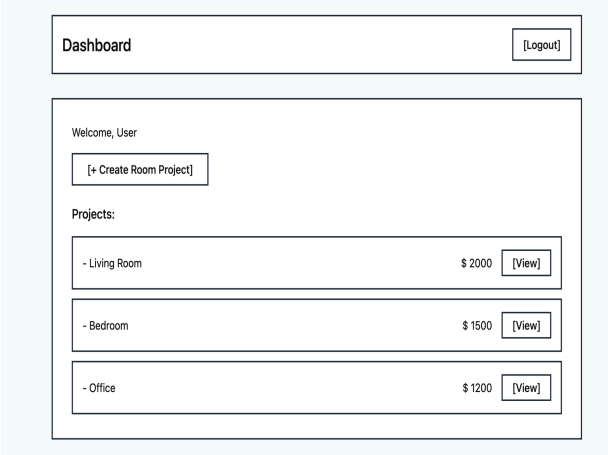
FR25 — AI Tag Suggestions
Future versions may include automated tag generation using computer vision.

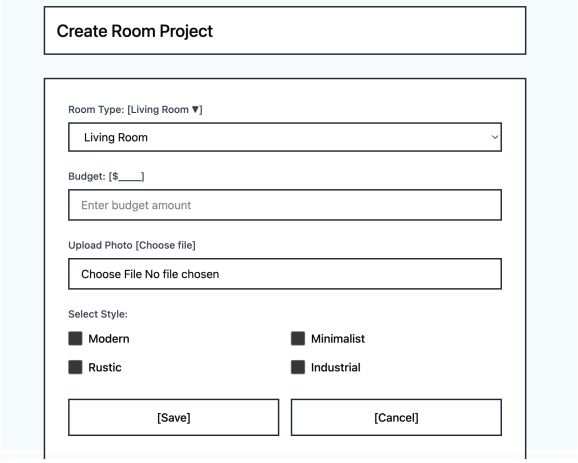
---

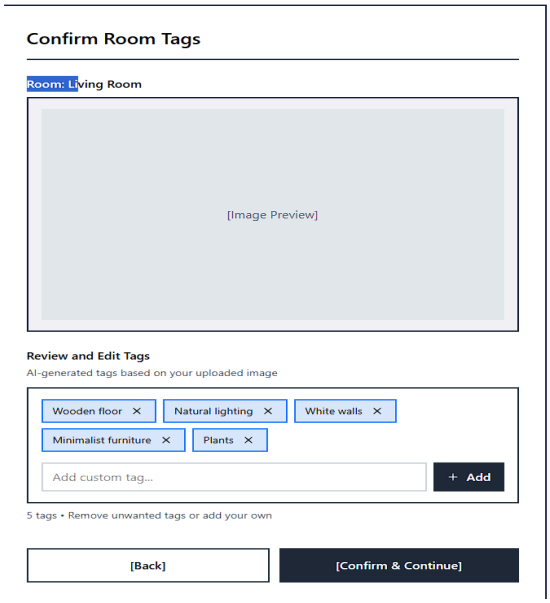
3. UI/UX Wireframes

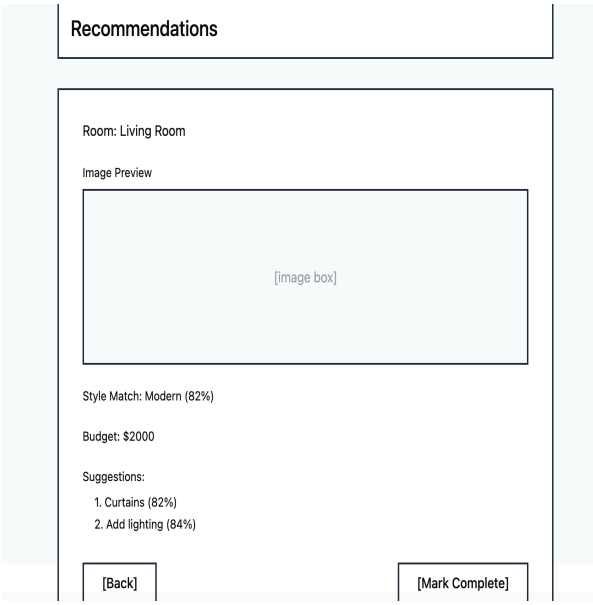
https://repeat-tile-19738019.figma.site/








Confirm Room Tags
Room: Living Room
[Image Preview]
Review and Edit Tags
AI-generated tags based on your uploaded image
Wooden floor
Natural lighting
White walls
Minimalist furniture
Plants
Add custom tag...
Add
5 tags • Remove unwanted tags or add your own
[Back]
[Confirm & Continue]


Recommendations
Room: Living Room
Image Preview
[image box]
Style Match: Modern (82%)
Budget: $2000
Suggestions:
1. Curtains (82%)
2. Add lighting (84%)
[Back]
[Mark Complete]

4. High-Level System Design

4.1 Database Architecture

The Home4U database schema is designed using a normalized relational structure. The schema includes entities for users, room projects, styles, tags, recommendations, and product items.

Initial Database Requirements

The database design is driven by the Priority 1 functional requirements of the system, including user authentication, room project creation, project retrieval, style browsing, and recommendation generation. The database must support storing user accounts, associating users with their room projects, and maintaining relationships between room features and design styles through tags. It must also support retrieving recommendations based on computed similarity scores while ensuring that each user can only access their own data.

DBMS Selection and Justification

For the current vertical prototype, the system uses SQLite due to its simplicity, lightweight setup, and ease of integration during development. SQLite allows rapid iteration without requiring
complex configuration. However, for production deployment, the system will migrate to PostgreSQL. PostgreSQL is better suited for handling concurrent users, larger datasets, and more complex queries. It also provides stronger data integrity, indexing capabilities, and support for replication, making it more appropriate for a scalable production environment.

Media Storage Strategy

In the current implementation, uploaded room images are stored locally in an uploads directory on the server, and their file paths are stored in the database. While this approach is sufficient for a single-instance prototype, it does not scale well in distributed environments. In future versions, media storage will be migrated to AWS S3, which provides scalable, durable, and centralized storage accessible by multiple backend instances. This change will improve system reliability and support horizontal scaling.

Data Integrity and Relationships

The database enforces relationships between entities using foreign keys to maintain data consistency. For example, each RoomProject is associated with a valid User, and recommendations are linked to specific projects. Join tables such as RoomTag and StyleTag ensure proper mapping between tags, styles, and room projects. These relationships support

accurate similarity scoring and recommendation generation while maintaining referential integrity across the system.

Future Improvements

Future enhancements to the database architecture include adding indexing to frequently queried fields such as style names and tags, implementing caching mechanisms for faster retrieval of recommendation data, and enabling database replication for improved availability and fault tolerance. These improvements will allow the system to handle increased user traffic and data volume more efficiently.

Entity Relationship Diagram

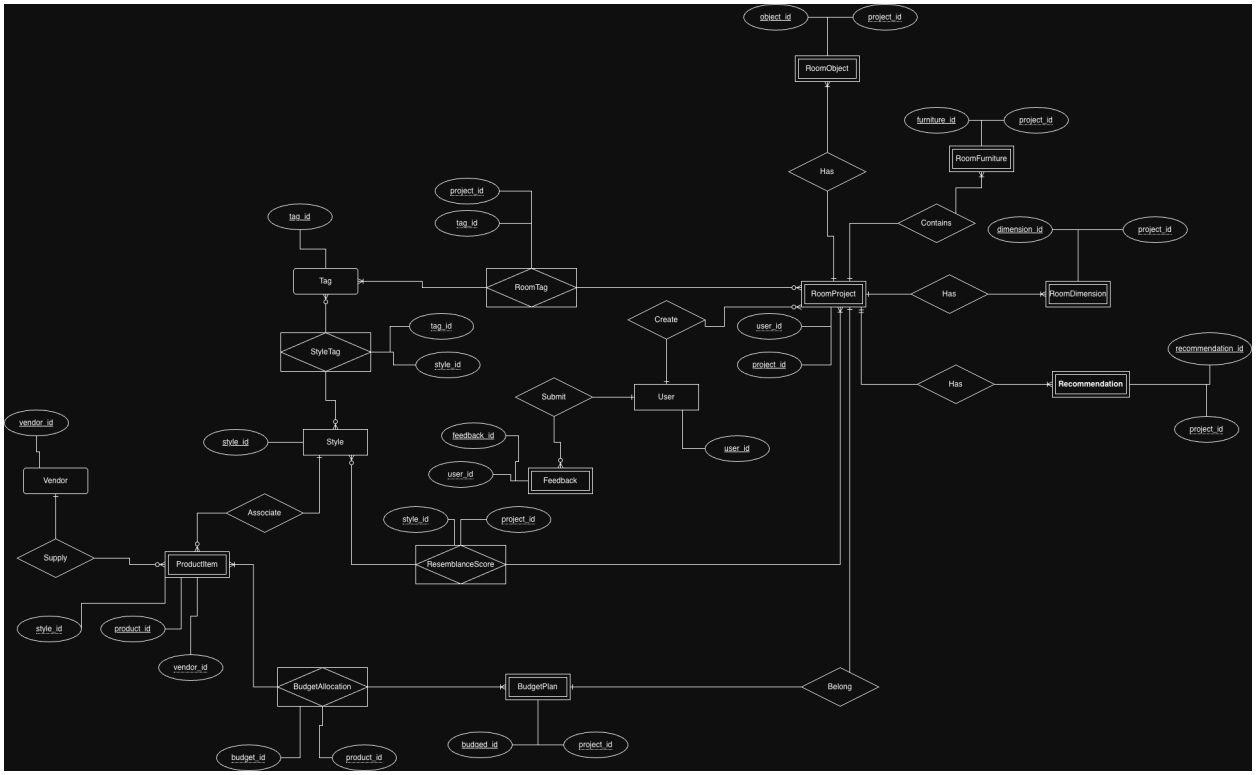
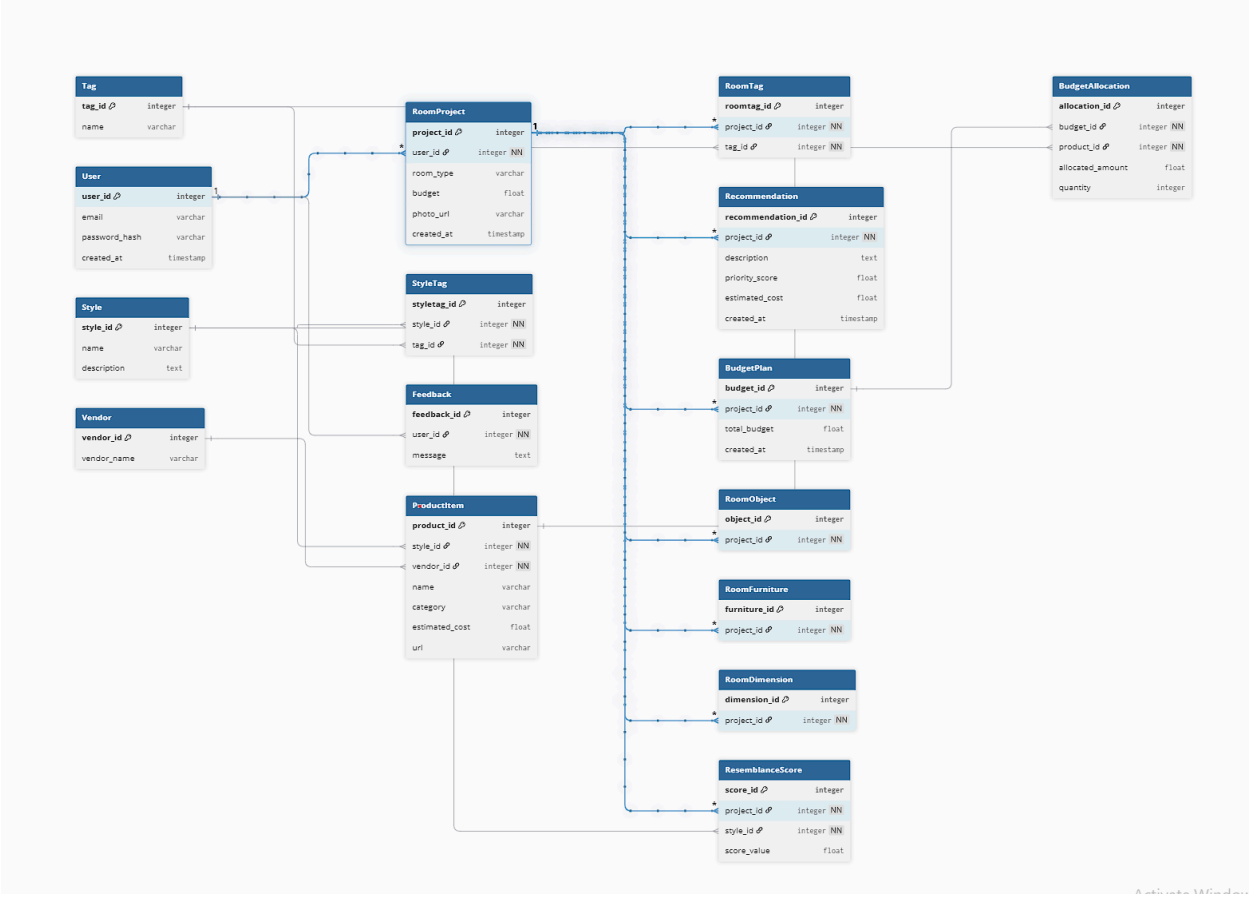


Figure 1 — Home4U Database Entity Relationship Diagram



4.2 Backend Architecture

The Home4U backend is designed as a modular service-oriented system that supports user authentication, room project management, style and tag retrieval, AI-assisted room analysis, resemblance-score computation, and recommendation generation. The backend is implemented using **FastAPI** with **Python 3.12** and communicates with a **PostgreSQL** relational database for persistent storage. This architecture separates request handling, business logic, and data persistence in order to improve maintainability, testability, and scalability.

At a high level, the backend consists of five major layers: **API/controller layer**, **service layer**, **data access layer**, **integration layer**, and **database layer**. The API/controller layer exposes

REST endpoints for core application features such as user registration, login, project creation, room image upload, tag retrieval, and recommendation generation. These controllers validate incoming requests and delegate business operations to service classes instead of implementing logic directly.

The **service layer** contains the core application logic. This includes user authentication, project ownership validation, room-tag generation, similarity-score calculation, and recommendation prioritization. The service layer acts as the central coordinator between controllers, repositories, and third-party integrations. This separation ensures that the backend remains organized as the project grows and that business rules are implemented consistently across endpoints.

The **data access layer** is responsible for reading from and writing to PostgreSQL. It manages application entities such as User, RoomProject, Style, Tag, RoomTag, StyleTag, ResemblanceScore, Recommendation, ProductItem, BudgetPlan, and BudgetAllocation. This layer isolates persistence logic from higher-level business logic, which makes the system easier to test and update when database structures evolve.

The **integration layer** supports external AI-powered room analysis. When a user uploads a room image, the backend sends the image to an AI vision service to obtain descriptive room tags. These tags are then normalized into the internal application format and stored for later comparison against predefined style tags. This integration allows the system to transform raw image input into structured metadata that can be used by the recommendation engine.

The **recommendation workflow** begins when a user selects or uploads a room image for a project. The backend retrieves the room project, extracts or reuses room tags, compares them against style tags, computes resemblance scores, and generates prioritized recommendations. Recommendations are then stored in the database and returned to the frontend for display. This workflow allows the system to provide explainable, data-driven design guidance tailored to the user's selected style and room characteristics.

To improve software structure, the backend applies several design patterns:

- # MVC (Model-View-Controller):

  In Home4U, MVC is reflected in the way the application is split between the frontend, backend routes, and persistent data.
  - The **Model** includes entities such as User, RoomProject, Style, Tag, RoomTag, Recommendation, and BudgetPlan.
  - The **View** is the React frontend, where users register, log in, upload room images, browse styles, and view recommendations.
  - The **Controller** is implemented through FastAPI route handlers such as project endpoints, authentication endpoints, and recommendation endpoints.

For example, when a user uploads a room image:

1. The React frontend sends the request.
2. The FastAPI controller receives the request.
3. The controller calls the appropriate backend service.
4. The service updates or reads model data.
5. The result is returned to the frontend for display.

UML Diagram:

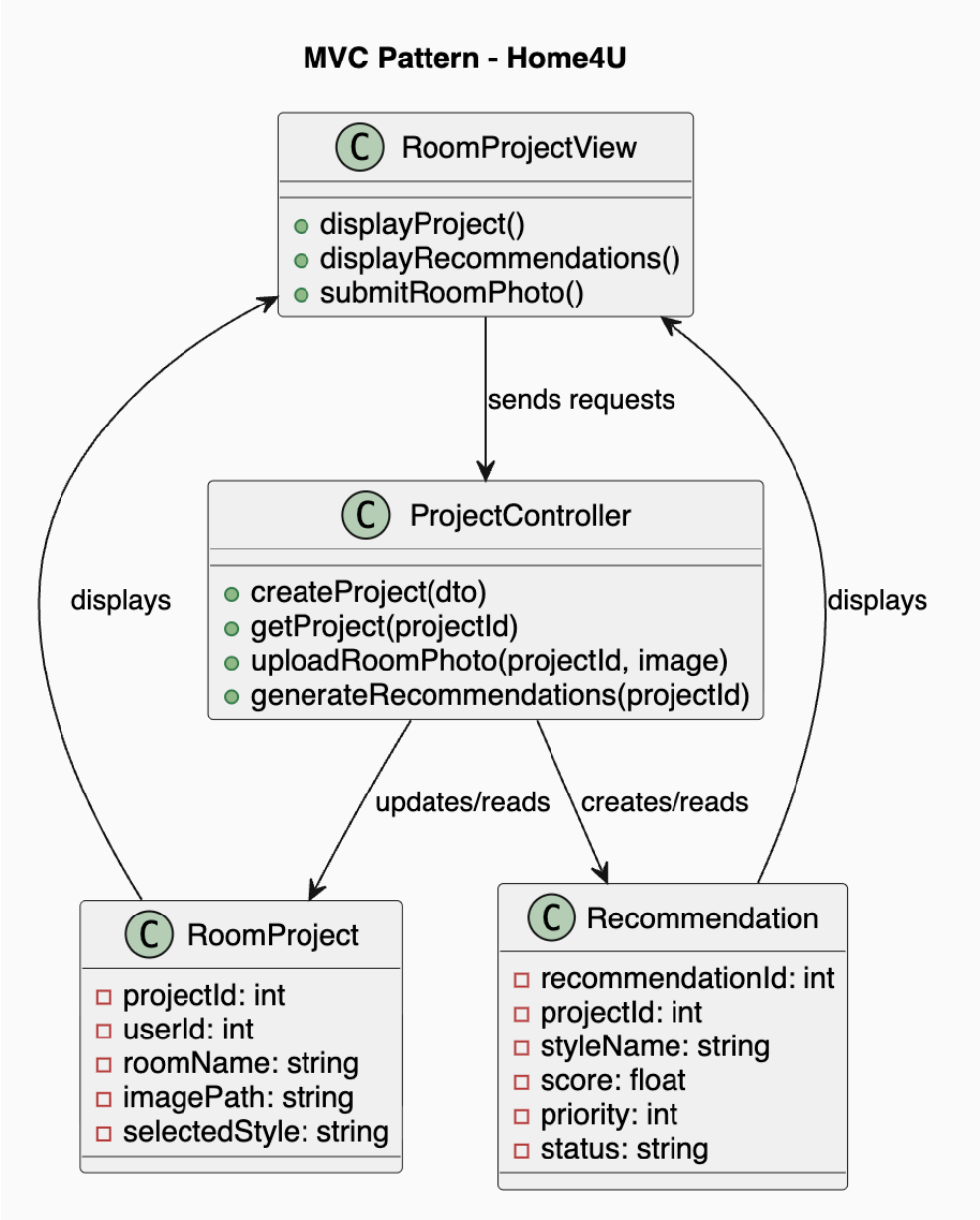


Figure 2 — Home4U MVC Pattern

- # Facade Pattern:

The **Facade** pattern provides a single, simplified interface to a larger and more complex subsystem. Instead of forcing one component to directly interact with many other classes, the facade hides that complexity behind one entry point.

Use in Home4U

This pattern is especially useful for the **recommendation workflow**, because recommendation generation is not a single simple action. It involves several steps:

1. Retrieve the room project from the database
2. Get the uploaded room image
3. Extract room tags from the image
4. Retrieve candidate interior styles
5. Compare room tags against style tags
6. Compute resemblance scores
7. Rank or prioritize the recommendations
8. Save the recommendations
9. Return them to the frontend

Without a facade, the controller would need to call many different services directly. That would make the controller too large and too complicated.

With a facade, the controller only calls something like:

```
RecommendationFacade.generateRecommendations(projectId)
```

Then the facade handles the rest.

UML Diagram:

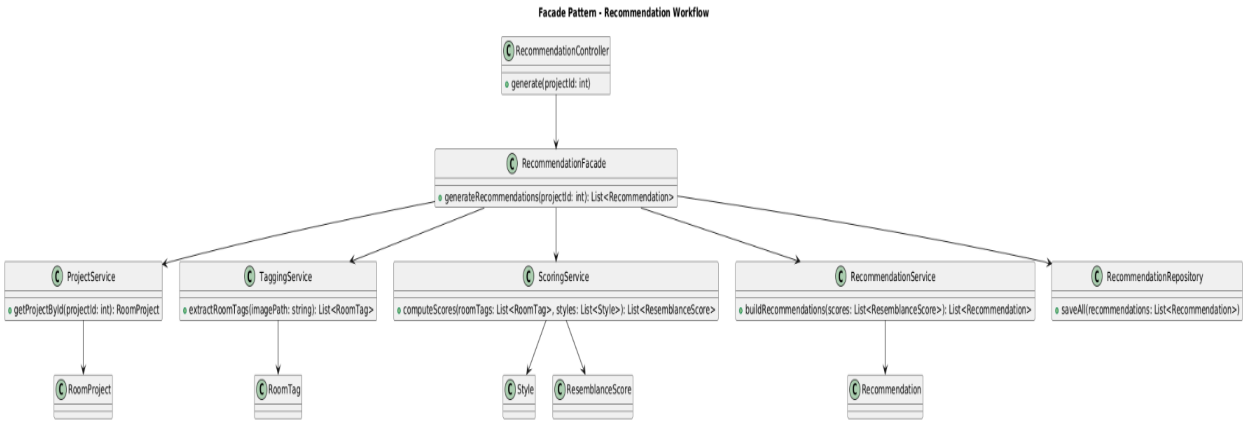


Figure 3 — Home4U Facade Pattern

● Factory Pattern:

The **Factory pattern** is used to centralize and abstract the creation of objects. Instead of instantiating objects directly using constructors, a factory class determines which concrete implementation to create based on input parameters or system context. This allows the system to depend on interfaces rather than specific implementations, improving flexibility and maintainability.

Use in Home4U

In Home4U, the Factory pattern is applied through a **ScoringServiceFactory**, which creates the appropriate scoring algorithm for recommendation generation. Since the system compares room tags against style tags and computes resemblance scores, different scoring approaches may be used depending on application needs.

The scoring services include:

- **WeightedTagScoringService** — computes similarity primarily using weighted room and style tag matches
- **BudgetAwareScoringService** — adjusts scoring based on budget compatibility in addition to tag similarity
- **HybridScoringService** — combines multiple factors such as tag overlap, budget fit, and priority weighting

When the backend needs to compute resemblance scores, it does not instantiate one of these classes directly. Instead, it calls:

```
ScoringServiceFactory.createService(mode)
```

The factory evaluates the requested mode and returns the appropriate implementation of the `ScoringService` interface.

How It Works

1. A recommendation request reaches the controller or recommendation facade.
2. The controller/facade calls `ScoringServiceFactory.createService(mode)`.
3. The factory selects the appropriate scoring implementation.
4. The selected scoring service computes resemblance scores.
5. Those scores are then used to rank and generate recommendations.

This ensures that the rest of the system depends only on the `ScoringService` interface and remains independent of specific scoring implementations.

UML Diagram:

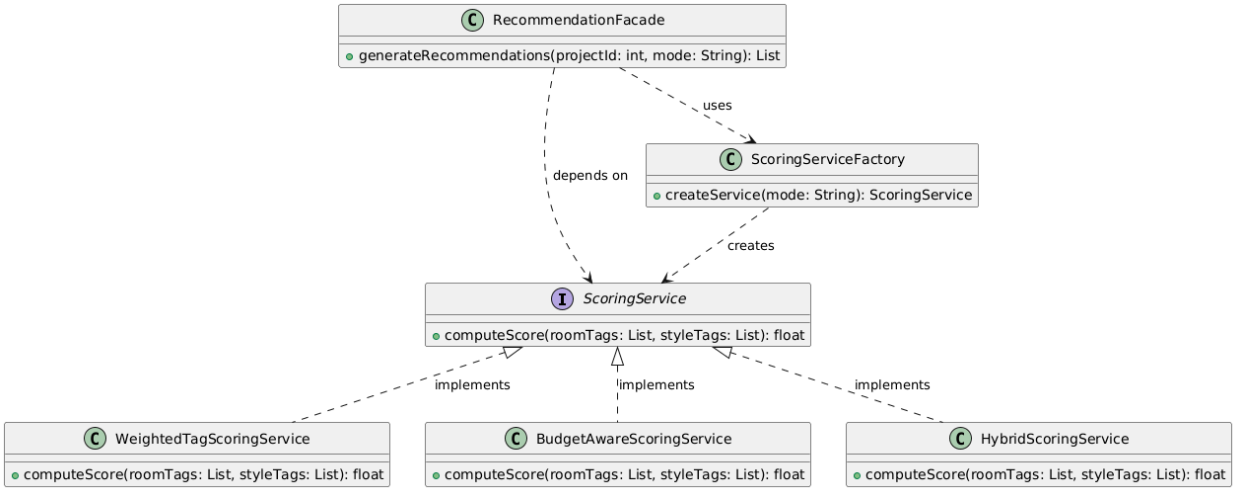


Figure 4 — Home4U Factory Pattern

• Builder Pattern:

The **Builder pattern** is used to construct complex objects step by step. Instead of creating an object through one large constructor with many parameters, the object is assembled gradually using a builder class. This separates the construction process from the final representation of the object and makes the code easier to read, maintain, and extend.

Use in Home4U

In Home4U, the Builder pattern is applied to the creation of a **Recommendation** object. A recommendation is not a simple data record; it is built from multiple pieces of information produced during the recommendation workflow.

A recommendation may include:

- **styleName** — the matched interior style

- **score** — the computed resemblance score
- **priority** — the ranking priority of the recommendation
- **matchedTags** — the tags shared between the room and the style
- **budgetFit** — whether the recommendation aligns with the user's budget
- **explanation** — a short explanation describing why the recommendation was generated

Since these values may be generated at different stages of the backend pipeline, constructing the `Recommendation` object through a builder is more appropriate than passing all attributes directly into one constructor.

In this design, the backend uses a `RecommendationBuilder` interface and a `ConcreteRecommendationBuilder` implementation to assemble recommendation objects step by step. The `RecommendationService` acts as the client that coordinates the construction process and requests the final object through the builder.

How It Works

1. The recommendation workflow computes intermediate values such as style match, score, and budget compatibility.
2. The `RecommendationService` creates or receives a `RecommendationBuilder`.
3. The builder is called step by step to set the required fields, such as style name, score, priority, and matched tags.
4. After all required values are assigned, the builder's `build()` method returns the completed `Recommendation` object.
5. The final recommendation is then stored or returned to the frontend.

This process keeps object construction organized and makes the code easier to expand if new recommendation attributes are added later.

UML Diagram:

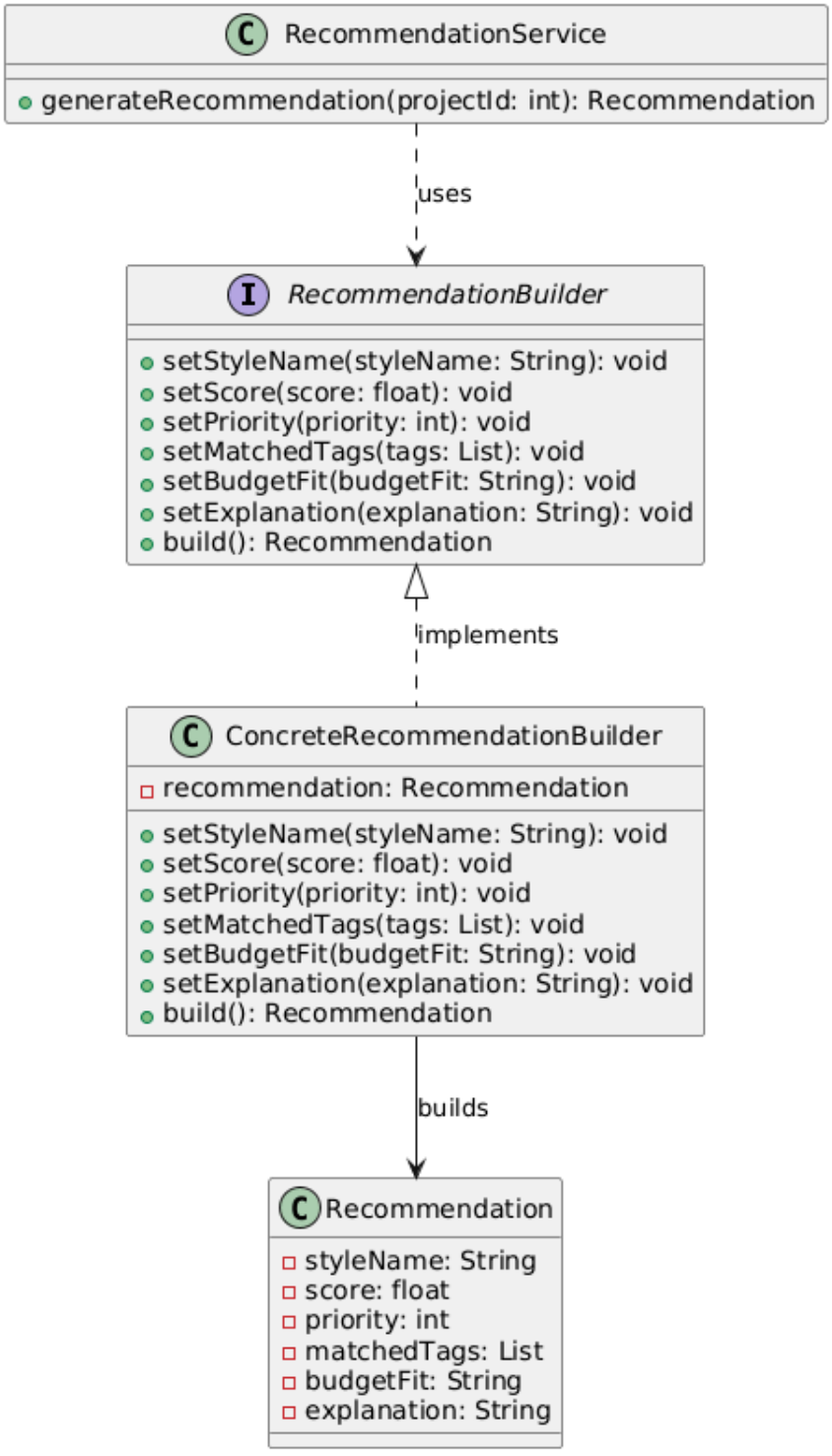


Figure 5 — Home4U Builder Pattern

The Recommendation Workflow:

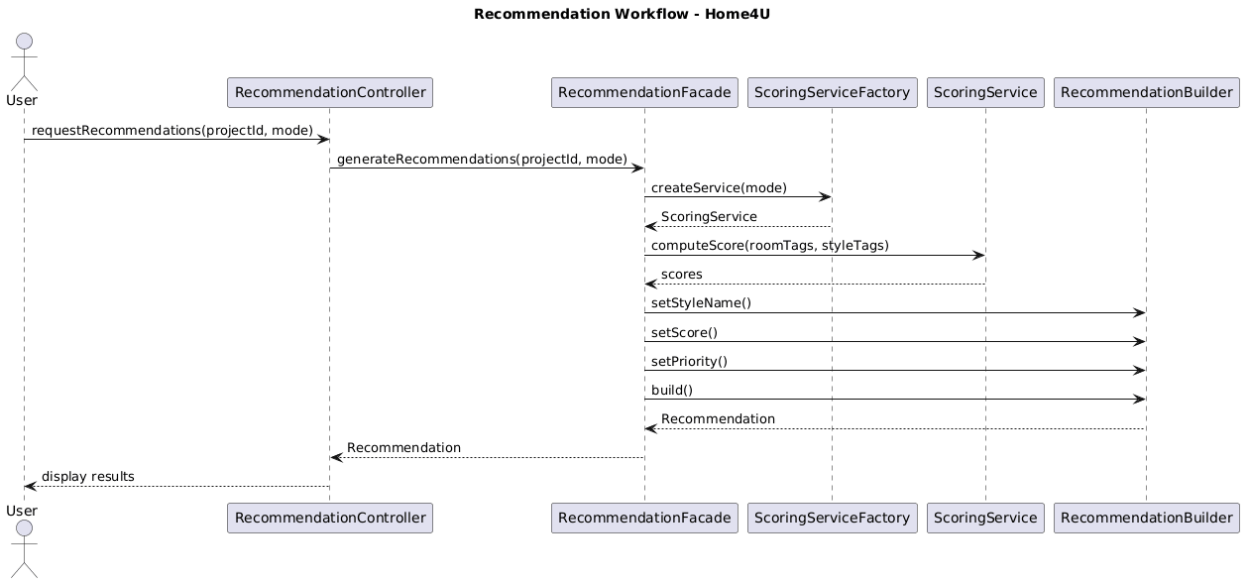


Figure 6 — Home4U Recommendation Workflow

From a security perspective, the backend validates user credentials, restricts access to authenticated endpoints, and enforces project ownership so users can only access their own room projects and recommendations. Basic input validation is performed at the API boundary, and sensitive configuration such as API keys and database credentials is stored outside the source code using environment variables. These practices support the milestone requirement for secure handling of credentials, access control, and awareness of common vulnerabilities.

The architecture is designed to support future scalability. New recommendation algorithms, additional AI analysis providers, and more advanced product-suggestion workflows can be added without requiring major changes to the controller layer. By separating backend responsibilities into distinct layers and components, Home4U remains flexible enough to support both current milestone requirements and future expansion.

5. Team Contributions

Caleb Ponce — Team Lead, Frontend Development, Integration (9)
Caleb Ponce led the overall coordination of the team and ensured alignment across all milestone requirements. He contributed to frontend development, focusing on improving UI consistency, usability, and overall user experience. Caleb also worked on integrating frontend components with backend services to ensure smooth system functionality and cohesive interaction across the platform.

Mason Lee — Frontend/UI Development (8)
Mason Lee collaborated closely on frontend development, focusing on maintaining design consistency and improving the usability of the application. His work ensured that user interactions across the platform were intuitive, visually consistent, and aligned with the system's design goals.

Christopher Quach — Documentation & System Organization (8)
Christopher Quach contributed to organizing and refining the project documentation. He ensured that all sections of the milestone were properly structured, clearly written, and aligned with project requirements, helping maintain clarity and completeness across the deliverables.

Tyler Morris — Diagrams, Workflows, QA Support (9)
Tyler Morris contributed to the creation and refinement of system diagrams, workflows, and structural representations of the application. He also supported quality assurance by verifying that system flows and architectural components were accurately represented and aligned with implementation.

Dias Almat — Diagrams, Workflows, System Design Support (9)
Dias Almat played a key role in developing and refining diagrams and workflow representations, particularly in illustrating system architecture and design patterns. He ensured that technical concepts such as workflows and system interactions were clearly visualized and aligned with the backend design.